

Agora — Τεκμηρίωση

Agentic Sales Support System για χονδρέμπορο ηλεκτρολογικού και δικτυακού υλικού. Τελική εργασία για το πρόγραμμα «AI for Developers» του Οικονομικού Πανεπιστημίου Αθηνών.

1. Τίτλος, περιγραφή και σκοπός

Το Agora είναι σύστημα υποστήριξης πωλήσεων για χονδρέμπορο ηλεκτρολογικού και δικτυακού υλικού, χτισμένο πάνω σε γλωσσικό μοντέλο. Ο πωλητής γράφει σε φυσική γλώσσα τι ζήτησε ο πελάτης και παίρνει πίσω έως πέντε σενάρια προσφοράς, κοστολογημένα από τον “πραγματικό” κατάλογο, το “πραγματικό” απόθεμα και τον “πραγματικό” τιμοκατάλογο, και ελεγμένα ξανά απέναντι στη βάση πριν εμφανιστούν. Δίπλα στις προσφορές υπάρχει μια δεύτερη διαδρομή για τις ερωτήσεις που ανακύπτουν γύρω από μια προσφορά: τι απόθεμα έχει ένας κωδικός, τι υπάρχει σε μια κατηγορία κάτω από μια τιμή, πόσα χρόνια εγγύηση έχει ένα UPS, μέχρι ποια ώρα φεύγει αυθημερόν μια παραγγελία. Το μοντέλο καταλαβαίνει την πρόταση και γράφει την απάντηση, δεν υπολογίζει ποτέ τιμή, απόθεμα ή ημερομηνία, και ό,τι γράφει ελέγχεται πριν φτάσει στην οθόνη. Τον τελικό λόγο για το τι θα φύγει προς τον πελάτη τον έχει ο πωλητής.

Σκοπός του εργαλείου είναι να εξυπηρετήσει τον πωλητή, που κάθεται στο τηλέφωνο ή στον πάγκο ενός καταστήματος χονδρικής και πρέπει να απαντήσει γρήγορα σε αιτήματα σαν το «θέλω δέκα τροφοδοτικά 80+ Gold σε ATX, τουλάχιστον 1000W». Για να δώσει σωστή προσφορά πρέπει να ξέρει ποια προϊόντα καλύπτουν το αίτημα, τι κοστίζει το καθένα, σε ποια αποθήκη βρίσκεται και πόσα κομμάτια, τότε παραδίδεται από εκεί στον προορισμό του πελάτη, τι έκπτωση όγκου δικαιούται η παραγγελία και αν μπορεί να την υπογράψει μόνος του, και αν ο προμηθευτής ενός προϊόντος που πρέπει να παραγγελθεί δεσμεύεται για επείγουσες παραδόσεις. Στο υποθετικό μας σενάριο οι πληροφορίες αυτές ζουν στο ERP, στο σύστημα αποθήκης, στον τιμοκατάλογο και σε τέσσερα έγγραφα πολιτικής, και ο πωλητής τις συνθέτει στο μυαλό του, υπό πίεση, για κάθε αίτημα στην πραγματικότητα ως έχει αυτήν την στιγμή. Το πρόβλημα που λύνει το Agora είναι αυτή η σύνθεση: κατανόηση της πρότασης, ανάκτηση των υποψηφίων, υπολογισμός των σεναρίων με απλή αριθμητική πάνω στον κατάλογο, επαλήθευση κάθε ποσού από τη βάση, και μόνο τότε μία σύσταση από το μοντέλο. Αυτή η αλυσίδα είναι ο λόγος που καμία τιμή στην οθόνη δεν προήλθε από το μοντέλο.

2. Σενάριο χρήσης και λειτουργικές απαιτήσεις

Ο πωλητής συνδέεται με όνομα χρήστη και κωδικό και βρίσκεται μπροστά σε δύο καρτέλες. Στην πρώτη γράφει το αίτημα του πελάτη όπως το άκουσε, στα ελληνικά ή στα αγγλικά: «Σαράντα καλώδια CAT6a θωρακισμένα, τουλάχιστον 20 μέτρα, τα θέλουμε άμεσα». Διαλέγει από ποιο κατάστημα σηκώνεται η παραγγελία, γιατί από αυτό εξαρτώνται τα μεταφορικά και οι ημέρες, και δηλώνει αν η παραγγελία επιβεβαιώνεται πριν τις 13:00, που είναι η προϋπόθεση της αυθημερόν παράδοσης. Στη δεύτερη καρτέλα κάνει ερωτήσεις σε μορφή συνομιλίας, και κάθε συνομιλία αποθηκεύεται ώστε μια ακόλουθη ερώτηση να έχει γνώση του τι προηγήθηκε στην συνομιλία.

Ο πρώτος δρόμος είναι η προσφορά.

Το σύστημα οφείλει:

- να διαβάσει την πρόταση σε απαιτήσεις που ο κατάλογος μπορεί να ελέγξει, κατηγορία, ποσότητα, τεχνικές συνθήκες με τελεστή, προϋπολογισμό, άμεση διαθεσιμότητα
- να βρει τα προϊόντα που τις ικανοποιούν και, όταν δεν τις ικανοποιεί κανένα, να πει ποια είναι μία παραχώρηση μακριά
- να χτίσει έως πέντε σενάρια, το φθηνότερο που καλύπτει το αίτημα, το πιο κοντινό, το καλύτερο για τα χρήματα, το γρηγορότερο και ένα σκαλοπάτι πάνω, κοστολογημένα με την έκπτωση όγκου, τα μεταφορικά και τις ημέρες όπως τα ορίζουν τα έγγραφα της εταιρείας
- να ελέγξει το καθένα απέναντι στη βάση όπως είναι εκείνη τη στιγμή και να αποσύρει ό,τι θα παραπλανούσε τον πελάτη
- και να προτείνει ένα, με αιτιολόγηση προς τον πωλητή και κείμενο προς τον πελάτη.

Ο στόχος είναι ένα αίτημα που ο κατάλογος δεν μπορεί να απαντήσει γυρίζει με τον λόγο, όχι με σφάλμα, γιατί το «δεν έχουμε τίποτα να πουλήσουμε» είναι απάντηση.

Ο δεύτερος δρόμος είναι η ερώτηση.

Εκεί η ακολουθία των βημάτων δεν είναι γνωστή εκ των προτέρων: άλλη ερώτηση θέλει το απόθεμα ενός κωδικού, άλλη ένα τμήμα από τα έγγραφα, και μερικές θέλουν δύο tools μαζί. Το σύστημα αφήνει το μοντέλο να διαλέξει ποιο tool θα καλέσει και πόσες φορές, μέσα σε όριο γύρων, και ελέγχει την απάντηση απέναντι σε ό,τι επέστρεψαν τα tools σε αυτόν τον γύρο. Μια απάντηση που δεν τεκμηριώνεται γυρίζει μία φορά για διόρθωση και μετά αντικαθίσταται από άρνηση που κατονομάζει τι δεν τεκμηριώθηκε.

Και στους δύο δρόμους η απόφαση είναι του πωλητή, όχι του συστήματος. Η οθόνη δείχνει όλες τις προσφορές δίπλα δίπλα και όχι μόνο την προτεινόμενη, δείχνει τα σενάρια που ο έλεγχος απέσυρε μαζί με τον λόγο, και κάτω από κάθε απάντηση της συνομιλίας γράφει από ποιο tool προήλθε. Το σύστημα δεν στέλνει τίποτα σε πελάτη, δεν δεσμεύει απόθεμα και δεν εγκρίνει έκπτωση. Οι υποστηρικτικές απαιτήσεις είναι σύνδεση με κωδικό, ιστορικό συνομιλιών ανά πωλητή, καταγραφή κάθε κλήσης προς μοντέλο, και ένας κατάλογος που μεγαλώνει εν λειτουργία.

3. Τεχνολογίες και γιατί τις διάλεξα

FastAPI. Και οι δύο υπηρεσίες είναι εφαρμογές FastAPI: τα schemas είναι μοντέλα Pydantic, οπότε η επικύρωση, το Swagger και η μορφή της απάντησης βγαίνουν από τον ίδιο ορισμό, και το dependency injection κάνει το «ποιος ρωτάει» ένα όρισμα πάνω στο route.

SQLite με SQLAlchemy. Ο κατάλογος είναι μία βάση SQLite, το data/catalog.db, committed και γεμάτη με τα 226 προϊόντα, ώστε ένα clone να τρέχει χωρίς να στήσει τίποτα. Το προτίμησα από την PostgreSQL, που θα ήταν η επιλογή σε παραγωγή αλλά εδώ θα πρόσθετε έναν server για μία μηχανή.

Chroma. Ο vector store είναι τοπική, persistent εγκατάσταση Chroma με δύο collections, products και policies. Κρατά metadata δίπλα σε κάθε διάνυσμα και δέχεται φίλτρο where πάνω τους πριν την αναζήτηση, που είναι ό,τι χρειάζεται η υβριδική ανάκτηση.

text-embedding-3-small. Τα διανύσματα των 226 καρτών και των 66 αποσπασμάτων είναι από το text-embedding-3-small της OpenAI, 1.536 διαστάσεις. Το προτίμησα από το text-embedding-3-large γιατί κοστίζει 0,02 έναντι 0,13 δολαρίων ανά εκατομμύριο tokens και γιατί σε αυτό το corpus η ανάκτηση με το μικρό μετρήθηκε στο 92% στα πέντε αποτελέσματα και 99% στα δέκα.

LangGraph. Και οι δύο διαδρομές είναι μηχανές καταστάσεων LangGraph. Το προτίμησα από χειρόγραφο βρόχο γιατί ο γράφος ελέγχεται ως δομή, και από τους έτοιμους agents της LangChain γιατί εκεί η ακολουθία των βημάτων ανήκει στη βιβλιοθήκη, ενώ στην προσφορά η ακολουθία είναι το ίδιο το σχέδιο.

gpt-5.1 (και gpt-4o-mini στις μετρήσεις). Το γλωσσικό μοντέλο για την εξαγωγή, τη σύσταση και τις ερωτήσεις είναι το gpt-5.1, μέσω του OpenAI SDK, με temperature 0 και στις τρεις κλήσεις. Το έργο όμως χτίστηκε και μετρήθηκε πάνω στο gpt-4o-mini, που κοστίζει 0,15 και 0,60 δολάρια ανά εκατομμύριο tokens εισόδου και εξόδου έναντι 1,25 και 10,00 του gpt-5.1: με δεκάδες μετρήσεις των τριών εκτελέσεων, η διαφορά είναι η διαφορά ανάμεσα στο να μετράω και στο να μη μετράω, και το να διαβάσει μια πρόταση σε ένα schema και να γράψει τρεις πάνω σε έτοιμα δεδομένα δεν ζητούσε περισσότερο. Το gpt-4o-mini έγραψε και τα 226 κείμενα του e-shop και παραμένει η προεπιλογή του κώδικα όταν η μεταβλητή LLM_MODEL λείπει. Στο τέλος γύρισα τη ρύθμιση σε gpt-5.1, και με αυτό τρέχει σήμερα το σύστημα και παρήχθησαν τα παραδείγματα του αποθετηρίου. Η αλλαγή δεν είναι μετρημένη, και το γράφω επίτηδες: οι αριθμοί του κεφαλαίου 10 βγήκαν όλοι από το gpt-4o-mini και δεν τους επανέλαβα, οπότε ισχύουν για εκείνο και όχι για αυτό που τρέχει τώρα.

Gradio. Η οθόνη είναι εφαρμογή Gradio με Blocks: δύο καρτέλες, πίνακες, ένα accordion και ένα Chatbot. Το προτίμησα από το Streamlit γιατί το component της συνομιλίας και οι καρτέλες υπάρχουν έτοιμα.

Poetry. Οι εξαρτήσεις και το εικονικό περιβάλλον διαχειρίζονται από το Poetry, με package-mode απενεργοποιημένο γιατί το Agora είναι εφαρμογή και όχι βιβλιοθήκη. Το προτίμησα από ένα requirements.txt γιατί το poetry.lock κάνει το περιβάλλον αναπαραγώγιμο. **Η Python είναι η 3.14.4.**

ruff. Ένα εργαλείο για linting και μορφοποίηση, με τους κανόνες στο pyproject.toml: E4, E7, E9, F, I, B, UP, ISC σε 100 στήλες. Το προτίμησα από τον συνδυασμό flake8, isort και black γιατί κάνει και τα τρία σε ένα πέρασμα.

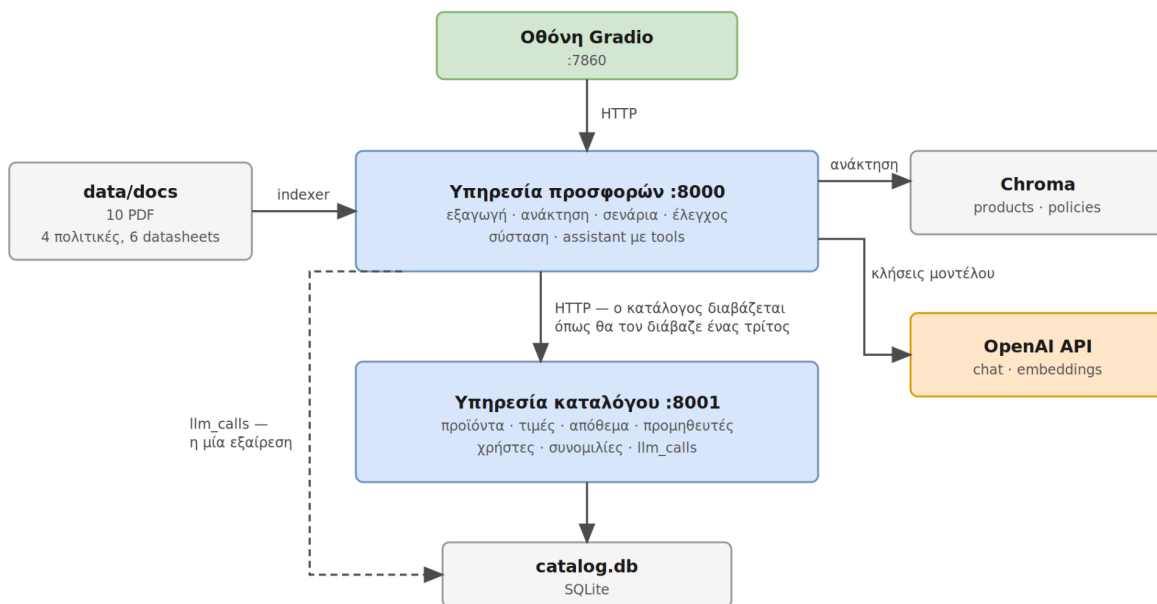
pytest. Τα 202 τεστ τρέχουν χωρίς δίκτυο και χωρίς κλειδί: κάθε κλήση προς μοντέλο παίρνεται από fixture, ο κατάλογος από βάση στη μνήμη και ο vector store από προσωρινό φάκελο με τα committed διανύσματα. Τι κάνει το μοντέλο στην πράξη το μετρούν ξεχωριστά τα εργαλεία του φακέλου tools.

JWT με bcrypt. Η σύνδεση είναι η ροή κωδικού του OAuth2: φόρμα με όνομα και κωδικό, πίσω ένα bearer token. Οι κωδικοί φυλάσσονται ως bcrypt hashes και το token είναι JWT υπογεγραμμένο με HS256 μέσω python-jose, διάρκειας 480 λεπτών. Το προτίμησα από sessions με cookie γιατί δύο υπηρεσίες πρέπει να συμφωνούν για το ποιος ρωτάει χωρίς να μοιράζονται πίνακα.

4. Αρχιτεκτονική και ροή δεδομένων

Το Agora είναι δύο υπηρεσίες FastAPI σε ένα αποθετήριο, που μιλούν μεταξύ τους μέσω HTTP. Η υπηρεσία καταλόγου, στη θύρα 8001, είναι η ιδιοκτήτρια των δεδομένων: της βάσης με τα προϊόντα, τις προδιαγραφές, το απόθεμα, τις τιμές και τους προμηθευτές, αλλά και των χρηστών, των συνομιλιών και του ημερολογίου των κλήσεων προς μοντέλο, μαζί με τα δύο μητρώα που διάβασα από τα έγγραφα της εταιρείας, το μοντέλο παράδοσης και το μοντέλο εκπτώσεων. Η υπηρεσία προσφορών, στη θύρα 8000, είναι η ιδιοκτήτρια της λογικής GenAI: εξαγωγή, ευρετήριο και ανάκτηση, σενάρια, επαλήθευση, σύσταση, και η διαδρομή των ερωτήσεων με τα tools της. Διαβάζει τον κατάλογο μέσα από έναν client httpx, όπως θα τον διάβαζε οποιοσδήποτε τρίτος, και δεν αγγίζει ποτέ προϊόντα, τιμές ή απόθεμα στη βάση. Η μία εξαίρεση είναι το ημερολόγιο των κλήσεων προς μοντέλο, που και οι δύο υπηρεσίες γράφουν στον ίδιο πίνακα μέσα από ένα κοινό module. Το όριο κρατά τον κατάλογο ελέγξιμο χωρίς μοντέλο, και κρατά τις τιμές και το απόθεμα στη βάση, από όπου διαβάζονται τη στιγμή που χτίζεται η προσφορά, ποτέ από το ευρετήριο, όπου θα ήταν φωτογραφία της περασμένης εβδομάδας. Η οθόνη Gradio, στη θύρα 7860, μιλά μόνο στην υπηρεσία προσφορών.

Agora — η αρχιτεκτονική

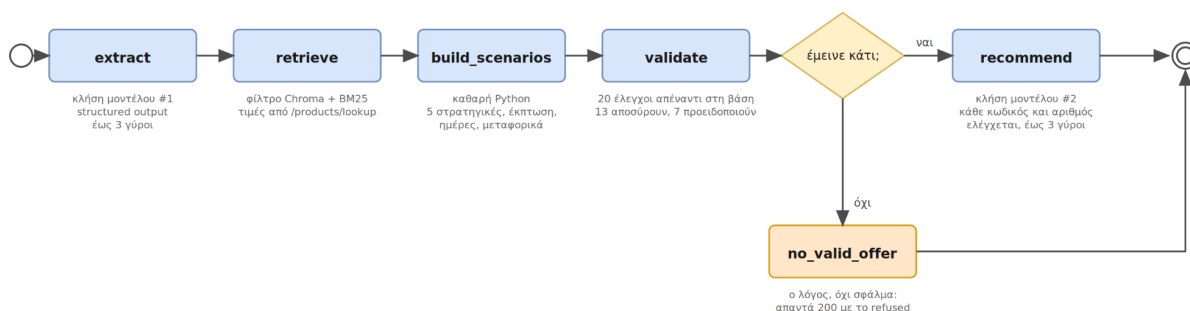


Το μοντέλο δεν βλέπει ποτέ τη βάση: τιμές, απόθεμα και ημερομηνίες τα διαβάζει ο κώδικας και τα υπολογίζει ο κώδικας.

Ένα αίτημα προσφοράς διασχίζει το σύστημα ως εξής. Το POST /offers/generate βάζει το αίτημα, το κατάστημα και τη σημαία των 13:00 σε μια αρχική κατάσταση και καλεί τον γράφο. Ο κόμβος extract στέλνει την πρόταση στο μοντέλο με structured output και επιστρέφει τις απαιτήσεις, ή τις ξαναστέλνει με τον λόγο της απόρριψης έως τρεις φορές. Ο κόμβος retrieve μεταφράζει τις συνθήκες σε φίλτρο πάνω στα metadata του Chroma, τρέχει BM25 και διανυσματική αναζήτηση πάνω στο ίδιο σύνολο επιλέξιμων προϊόντων, και κοστολογεί ό,τι βρήκε με μία κλήση στο /products/lookup. Ο κόμβος build_scenarios είναι καθαρή αριθμητική: κατανομή της ποσότητας σε αποθήκες, ημέρες, έκπτωση, μεταφορικά, σύνολο, ρίσκο. Ο κόμβος validate ξαναδιαβάζει τα ίδια προϊόντα από τον κατάλογο και ξαναυπολογίζει κάθε ποσό,

ημερομηνία και κατανομή, 13 έλεγχοι αποσύρουν το σενάριο όταν αποτύχουν και 7 ταξιδεύουν μαζί του ως προειδοποίηση. Η μία διακλάδωση είναι εδώ: αν κάτι επέζησε, ο κόμβος `recommend` δίνει στο μοντέλο τις προσφορές που πέρασαν και ελέγχει την απάντησή του κωδικό προς κωδικό και αριθμό προς αριθμό, αν όχι, ο κόμβος `no_valid_offer` γράφει σε ποιο βήμα κόπηκε το ίχνος.

Η προσφορά: σταθερό workflow, έξι κόμβοι



Η σειρά είναι γνωστή πριν διαβαστεί το αίτημα, γι' αυτό δεν τη διαλέγει το μοντέλο. Το μοντέλο καλείται δύο φορές: στην αρχή για να διαβάσει μία πρόταση, στο τέλος για να γράψει μία.

Το πού επιτρέπεται να αγγίξει το μοντέλο είναι η αρχιτεκτονική απόφαση που ορίζει το έργο. Στην προσφορά καλείται δύο φορές, στην αρχή για να διαβάσει μια πρόταση και στο τέλος για να γράψει μία, και ανάμεσά τους δεν έχει θέση: το φίλτρο, τα σενάρια, οι ημερομηνίες και οι εκπτώσεις βγαίνουν από κώδικα και μητρώα, και ο έλεγχος διαβάζει τη βάση. Στις ερωτήσεις το μοντέλο αποφασίζει ποιο tool θα καλέσει, αλλά τα tools επιστρέφουν γραμμές του καταλόγου και τμήματα εγγράφων όπως είναι, και η απάντηση διαβάζεται απέναντι στα αποτελέσματά τους πριν εμφανιστεί. Κάθε κλήση προς μοντέλο γράφεται στον πίνακα `llm_calls` με τα tokens της, οπότε το `GET /admin/usage` αθροίζει ολόκληρο τον λογαριασμό.

5. Τα δεδομένα

Ο κατάλογος είναι 226 προϊόντα σε έξι κατηγορίες: 58 καλώδια ρεύματος (POWER), 46 καλώδια δικτύου (DATA), 34 τροφοδοτικά (PSU), 28 UPS, 30 switches (NETWORK) και 30 ρευματολήπτες (CONNECTORS), πέντε μάρκες, τέσσερις αποθήκες, ATH-01 και ATH-02 στην Αττική, THE-01 στη Θεσσαλονίκη και PAT-01 στην Πάτρα, και τέσσερις προμηθευτές με χρόνο παραλαβής και δείκτη αξιοπιστίας: Ionia Electric 3 ημέρες και 0,96, Aegean Supplies 7 και 0,88, Balkan Cables 14 και 0,74, Hellenic Power Systems 5 και 0,92. Προμηθευτής κάτω από 0,80 δεν δεσμεύεται για επείγουσα παραγγελία. Κάθε προϊόν κουβαλά δύο κείμενα: τη λακωνική γραμμή του ERP με τις προδιαγραφές μέσα της και το κείμενο του e-shop, μία ή δύο προτάσεις για επαγγελματία αγοραστή, που είναι το κείμενο που αξίζει να εμφυτευθεί.

Το αρχείο προέλευσης είναι το data/raw/catalog.json. Το παράγει ένα script από σπόρο, ώστε ο ίδιος σπόρος να δίνει πάντα τον ίδιο κατάλογο. Τι είναι ένα προϊόν κάθε κατηγορίας, το ορίζει ένα μητρώο κατηγοριών που διαβάζουν ο παραγωγός, το API, το prompt της εξαγωγής και η οθόνη, ώστε μια νέα κατηγορία να μην μπορεί να ξεχαστεί σε ένα από τα τέσσερα. Πέντε προϊόντα δεν έχουν καθόλου εγγραφή αποθέματος, και ο κατάλογος τα ξεχωρίζει από τα 59 που έχουν εγγραφή με μηδέν: το άγνωστο απόθεμα δεν είναι μηδενικό. Τα κείμενα του e-shop τα έγραψε το μοντέλο, δέκα δέκα, και κάθε κείμενο κρατήθηκε μόνο αν οι ίδιοι parsers που διαβάζουν τη γραμμή του ERP μπορούσαν να ξαναδιαβάσουν από μέσα του κάθε προδιαγραφή που είχε διαλέξει ο κώδικας. Και τα 226 πέρασαν, με μέση επικάλυψη διατύπωσης 3% ανάμεσα στα 4.497 ζεύγη ίδιας κατηγορίας.

Το αρχείο φορτώνεται στη βάση μέσα από τον normalizer: κάθε τιμή περνά από τους parsers, όπου ζουν όλες οι κανονικές εκφράσεις του συστήματος. Ο κωδικός ανάγεται στη μορφή PWR-1007 από όποια γραφή κι αν ήρθε, η τιμή διαβάζεται είτε γράφτηκε «€ 134,20», «163.95» ή «145,17 EUR», η ημερομηνία διαβάζεται σε πέντε μορφές και ως σειριακός αριθμός του Excel, γιατί ένα κελί σε μορφή General γυρίζει το 2026-09-01 σε 46266. Στη βάση καταλήγουν 226 προϊόντα, 852 προδιαγραφές, 226 τιμές και 221 γραμμές αποθέματος.

Η ακαταστασία των πραγματικών εξαγωγών δεν είναι στο αρχείο, που είναι καθαρό, αλλά σε μια μονάδα που τη γράφει επίτηδες όταν το endpoint παραγωγής προσθέτει προϊόντα σε κατάλογο που ήδη τρέχει: το 12% των κωδικών χωρίς παύλα και με πεζά, το 3% των τιμών ως «N/A», το ποσό γραμμένο με τρεις τρόπους εναλλάξ, το 5% των γραμμών αποθέματος να μη φτάνει ποτέ. Οι γραμμές διαβάζονται πίσω από τον ίδιο normalizer, οπότε η κανονικοποίηση δουλεύει σε ζωντανά δεδομένα και όχι σε fixture. Ο ίδιος δρόμος δέχεται εισαγωγή CSV, σε όποια κωδικοποίηση και με όποιον διαχωριστή το έσωσε ένα ελληνικό Excel, και κάθε φόρτωση καταγράφεται ως run που μπορεί να αναιρεθεί.

Τα επιχειρησιακά έγγραφα είναι δέκα PDF στο data/docs. Τέσσερα είναι πολιτικές: η Πολιτική Εκπτώσεων με πέντε ενότητες, όπου ζουν τα κλιμάκια όγκου, τα όρια ανά κατηγορία και ποιος εγκρίνει τι, οι Όροι Παράδοσης με έξι, με τον πίνακα ημερών ανά αποθήκη και προορισμό, το cut-off των 13:00, τη μεταφορά μεταξύ αποθηκών και τα μεταφορικά με το όριο των 300 €, η Εγγύηση και Επιστροφές με έξι, και οι Όροι Πληρωμής με επτά. Τα άλλα έξι είναι datasheets, ένα ανά κατηγορία, με επτά ενότητες το καθένα, για έναν πραγματικό κωδικό: CON-1017, DAT-1028, PSU-1020, PWR-1045, SWT-1020 και UPS-1014. Κόβονται στις επικεφαλίδες τους σε 66 αποσπάσματα. Τα δύο μητρώα του κώδικα, παράδοση και εκπτώσεις, είναι μεταγραφές αυτών των εγγράφων, και ένα εργαλείο ελέγχει τους 43 αριθμούς τους και τις 32 ημερομηνίες που παράγουν απέναντι σε ό,τι διαβάζει το ίδιο μέσα από τα PDF.

6. Τα endpoints και οι υπηρεσίες

Η υπηρεσία καταλόγου, στο <http://localhost:8001>, εκθέτει είκοσι routes.

Μέθοδος	Διαδρομή	Τι κάνει
GET	/health	Έλεγχος ότι η υπηρεσία απαντά.
POST	/auth/login	Σύνδεση με φόρμα OAuth2, επιστρέφει bearer token.
GET	/products/search	Αναζήτηση με κατηγορία, ελάχιστο watt, ελάχιστο μήκος, ανώτατη τιμή, ελάχιστο απόθεμα.
POST	/products/lookup	Έως 100 προϊόντα μαζί, με προδιαγραφές, τιμή και απόθεμα ανά αποθήκη.
GET	/products/{sku}/stock	Πού φυλάσσεται ένα προϊόν και πόσα τεμάχια.
GET	/suppliers	Το μητρώο προμηθευτών.
GET	/stats	Τι περιέχει ο κατάλογος αυτή τη στιγμή.
POST	/admin/generate	Προσθέτει προϊόντα στον κατάλογο, από σπόρο και μέσα από τη «βρώμικη» εξαγωγή.
GET	/admin/runs	Οι προηγούμενες εκτελέσεις παραγωγής και εισαγωγής.
DELETE	/admin/runs/{request_id}	Αναιρεί μία εκτέλεση ολόκληρη.
POST	/admin/rebuild	Ξαναχτίζει τον κατάλογο από το αρχείο προέλευσης.
GET	/admin/usage	Τι έχουν κοστίσει τα μοντέλα, ανά υπηρεσία και σκοπό.
GET	/admin/import/template	Το πρότυπο CSV για εισαγωγή.
POST	/admin/import	Εισαγωγή συμπληρωμένου CSV, έως 5.000 γραμμές και 5 MB.
GET	/sql/schema	Οι πίνακες και οι στήλες που μπορεί κανείς να ρωτήσει.

Μέθοδος	Διαδρομή	Τι κάνει
POST	/sql/query	Ένα SELECT πάνω στον κατάλογο, μόνο για ανάγνωση.
POST	/conversations	Ανοίγει συνομιλία για έναν χρήστη (εσωτερικό).
GET	/conversations	Οι συνομιλίες ενός χρήστη, νεότερη πρώτη (εσωτερικό).
GET	/conversations/{id}	Μία συνομιλία με όλους τους γύρους της (εσωτερικό).
POST	/conversations/{id}/messages	Προσθέτει έναν γύρο σε συνομιλία (εσωτερικό).

Η υπηρεσία προσφορών, στο <http://localhost:8000>, εκθέτει έξι.

Μέθοδος	Διαδρομή	Τι κάνει
GET	/health	Έλεγχος ότι η υπηρεσία απαντά.
POST	/auth/login	Σύνδεση, προωθημένη στην υπηρεσία καταλόγου.
POST	/offers/generate	Από ένα αίτημα σε προσφορές και σύσταση.
POST	/assistant/ask	Μια ερώτηση, απαντημένη από αποτελέσματα tools.
GET	/assistant/conversations	Οι συνομιλίες του συνδεδεμένου χρήστη.
GET	/assistant/conversations/{id}	Μία συνομιλία με όλους τους γύρους της.

Το POST /offers/generate παίρνει το αίτημα, το κατάστημα, με προεπιλογή την Αθήνα, και αν η παραγγελία επιβεβαιώνεται πριν τις 13:00. Απαντά με τις απαιτήσεις όπως διαβάστηκαν, τις προσφορές που πέρασαν τον έλεγχο, τη σύσταση, κάθε σενάριο που χτίστηκε με την ετυμηγορία του, και ένα trace. Ένα αίτημα που ο κατάλογος δεν μπορεί να απαντήσει γυρίζει ως 200 με τον λόγο στο πεδίο refused, σφάλμα είναι μόνο ό,τι δεν είναι απάντηση, 503 όταν κάτι δεν απαντά και 502 όταν η εξαγωγή ή η σύσταση εξάντλησαν τους γύρους τους.

Το POST /assistant/ask παίρνει την ερώτηση και, αν η συνομιλία έχει ανοίξει, το αναγνωριστικό της, χωρίς αναγνωριστικό ανοίγει καινούργια στο όνομα του token. Στο μοντέλο δείχνονται οι δέκα τελευταίοι γύροι, ώστε το «και τι εγγύηση έχει;» να έχει υποκείμενο. Απαντά με το κείμενο, τις κλήσεις tools πίσω του, πόσους γύρους πήρε και αν είναι άρνηση. Η άρνηση είναι κι αυτή 200 και γράφεται στη συνομιλία, γιατί είναι απάντηση, ένα μοντέλο που δεν σταμάτησε να καλεί tools μέσα σε πέντε γύρους είναι 502 και δεν αφήνει τίποτα πίσω του. Οι διαδρομές των συνομιλιών διαβάζουν μόνο τις συνομιλίες του χρήστη που κρατά το token, και μια συνομιλία άλλου απαντά 404 και όχι 403, οι τέσσερις αντίστοιχες διαδρομές του καταλόγου είναι εσωτερικές.

Το /auth/login υπάρχει και στις δύο υπηρεσίες γιατί η οθόνη και το Swagger μιλούν σε μία

προέλευση. Η σύνδεση όμως ζει εκεί που ζουν οι χρήστες: ο κατάλογος έχει τον πίνακα users, άρα μόνο αυτός ελέγχει ένα bcrypt hash και εκδίδει token. Η υπηρεσία προσφορών προωθεί τη φόρμα όπως είναι, επιστρέφει το token αμετάβλητο, και από εκεί και πέρα το επαληθεύει μόνη της με το κοινό μυστικό.

7. Οι τεχνικές GenAI

Structured outputs. Καμία απάντηση μοντέλου δεν φτάνει στον κώδικα ως ελεύθερο κείμενο. Η εξαγωγή, στο `extraction.py`, ζητά από το μοντέλο ένα `schema` `Extracted` μέσα από το `structured-output` API της OpenAI: κατηγορία, ποσότητα, συνθήκες ως τριάδες κλειδί, τελεστής, τιμή, όρια τιμής και προϋπολογισμού, σημαία του άμεσου. Το αποτέλεσμα περνά στο `CustomerRequirements`, όπου οι `validators` κρίνουν: αριθμητική συνθήκη θετική, ετικέτα μέσα στη λίστα της, σύγκριση μόνο σε διατεταγμένες ετικέτες, κλειδί που ανήκει στην κατηγορία. Ό,τι απορρίπτεται γυρίζει στο μοντέλο ως μία πρόταση με τον λόγο, έως τρεις γύρους, και μετά το αίτημα απορρίπτεται αντί να μαντευτεί. Το ίδιο σχήμα, μοντέλο που προτείνει και κώδικας που κρίνει, τρέχει στα κείμενα του e-shop, με τους `parsers` ως κριτή, και στη σύσταση, με το `schema` `Written`.

Prompt engineering. Τα `prompts` είναι τρία `modules` στο `services/offer_service/prompts`: εξαγωγή, σύσταση, `assistant`. Το `prompt` της εξαγωγής παράγεται από το μητρώο κατηγοριών: οι κατηγορίες με τις προδιαγραφές τους, ποιες είναι αριθμητικές, ποιες λογικές, ποιες ετικέτες και με ποια διάταξη, τυπώνονται από τον ίδιο κώδικα που φτιάχνει το `schema` και το φίλτρο. Πάνω σε αυτό κάθονται οκτώ κανόνες, ένας πίνακας με τις λέξεις του επαγγέλματος, «θωρακισμένο» σε S/FTP, «gigabit» σε 1000, και έντεκα παραδείγματα. Κανένας κανόνας δεν μπήκε επειδή ακουγόταν σωστός: κάθε αλλαγή μετρήθηκε σε σταθερό σύνολο αιτημάτων, σε `temperature` 0, τρεις φορές, και κρατήθηκε μόνο αν μετακίνησε έναν αριθμό που κρατούσε και στις τρεις.

System prompts. Και τα τρία έχουν την ίδια ανατομία: ρόλος, είσοδος, έργο, αριθμημένοι κανόνες που λένε τι επιτρέπεται και πάνω απ' όλα τι απαγορεύεται, και αυτοέλεγχος πριν την απάντηση. Το `prompt` της σύστασης, στο `prompts/recommendation.py`, ορίζει τον αναγνώστη, τον πωλητή που αποφασίζει τι θα ακούσει ο πελάτης, και έντεκα κανόνες, ανάμεσά τους: κάθε αριθμός στις προσφορές ή στο αίτημα, χωρίς στρογγύλεμα, άθροισμα ή μετατροπή, μια προσφορά υστερεί μόνο εκεί που το λέει μια σημείωση, το κείμενο προς τον πελάτη είναι τρεις προτάσεις, προϊόν, σύνολο, πότε. Το `prompt` του `assistant`, στο `prompts/assistant.py`, λέει ότι δεν ξέρει τίποτα για αυτόν τον κατάλογο πέρα από ό,τι επέστρεψε ένα `tool` σε αυτόν τον γύρο, ότι οι προηγούμενοι γύροι δίνουν το θέμα και όχι αριθμούς, και ότι το «δεν βρέθηκε τίποτα» είναι απάντηση.

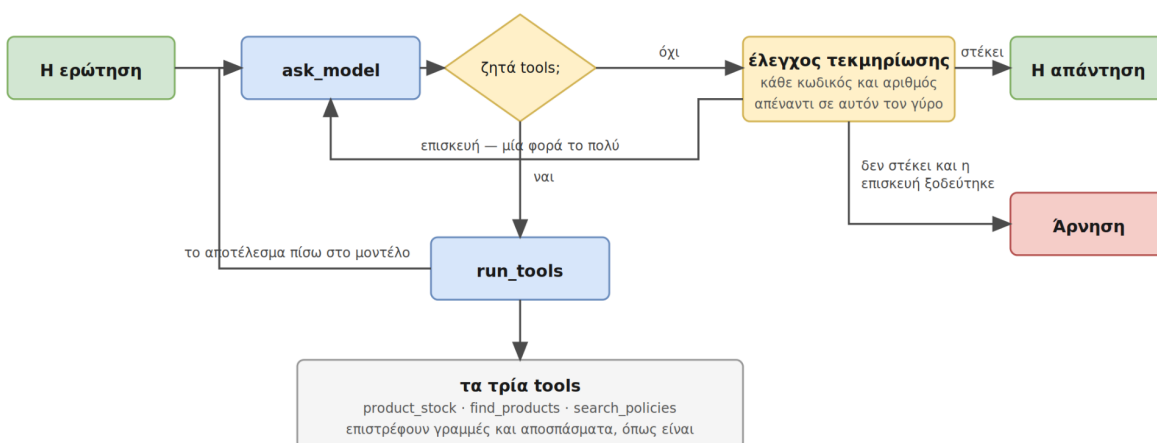
RAG. Το `rag/documents.py` γράφει μία κάρτα ανά προϊόν: κωδικός, κατηγορία και μάρκα, οι προδιαγραφές σε λέξεις, η γραμμή του ERP και το κείμενο του e-shop. Στα `metadata` μπαίνουν κωδικός, κατηγορία, μάρκα, προμηθευτής και κάθε προδιαγραφή, ποτέ η τιμή ή το απόθεμα. Το `rag/policies.py` κόβει τα δέκα PDF στις ενότητες τους και γράφει κάθε απόσπασμα με τον τίτλο του εγγράφου, τον κωδικό αν είναι `datasheet`, και την επικεφαλίδα μέσα στο κείμενό του, γιατί ένα απόσπασμα ανακτάται μόνο του. Η εμφύτευση γίνεται σε ένα σημείο, το `rag/embeddings.py`, και τα διανύσματα φυλάσσονται στο `data/raw` με ένα `blake2b` digest δίπλα στο καθένα, ώστε ο `indexer` να ζητά από το μοντέλο μόνο ό,τι άλλαξε.

Η αναζήτηση προϊόντων, στο `rag/retriever.py`, είναι υβριδική με συγκεκριμένη σειρά. Πρώτα το `rag/filters.py` μεταφράζει τις απαιτήσεις σε φίλτρο `where` πάνω στα `metadata`: ισότητα στην κατηγορία και μία συνθήκη ανά περιορισμό, όπου η σύγκριση σε διατεταγμένη ετικέτα, «τουλάχιστον CAT6a», γίνεται λίστα των τιμών που την ικανοποιούν. Η τιμή και το απόθεμα δεν μπαίνουν ποτέ στο φίλτρο. Πάνω στο σύνολο των επιλέξιμων τρέχουν δύο κατατάξεις, μία λεξική με BM25, στο `rag/lexical.py`, και μία διανυσματική, και οι δύο λίστες συγχωνεύονται με εναλλαγή, μία από την καθεμία, που εγγυάται σε ό,τι είναι πρώτο σε οποιαδήποτε από τις δύο θέση όχι χειρότερη από τη δεύτερη, δοκίμασα και `reciprocal rank fusion` και τη μέτρησα χειρότερη. Όταν το φίλτρο δεν αφήνει τίποτα, ο `retriever` ξαναρωτά με κάθε περιορισμό αφαιρεμένο μία φορά,

ενώνει τα σύνολα και σημειώνει στο trace ποιες παραχωρήσεις άνοιξαν τι. Αν το αίτημα ζητά υπερθετικό, «το μακρύτερο καλώδιο που έχετε», δεν τρέχει καμία κατάταξη: τα προϊόντα που πέρασαν το φίλτρο ταξινομούνται κατά την προδιαγραφή, γιατί αυτό είναι ερώτημα ORDER BY. Η αναζήτηση στα έγγραφα, `search_policies`, τρέχει τις ίδιες δύο κατατάξεις χωρίς φίλτρο πάνω και στα 66 αποσπάσματα.

Agents. Οι δύο διαδρομές είναι διαφορετικές μηχανές επίτηδες. Η προσφορά, στο `graph/builder.py`, είναι σταθερό workflow: έξι κόμβοι, `extract`, `retrieve`, `build_scenarios`, `validate`, `recommend` και `no_valid_offer`, με μία ακμή υπό συνθήκη μετά τον έλεγχο. Η σειρά είναι γνωστή πριν διαβαστεί το αίτημα και δεν υπάρχει λόγος να τη διαλέξει το μοντέλο, γιατί κάθε βήμα μετά το πρώτο δουλεύει πάνω σε αριθμούς που μόνο ο κώδικας εγγυάται. Ο assistant, στο `assistant/graph.py`, είναι βρόχος δύο κόμβων, `ask_model` και `run_tools`: το μοντέλο είτε ζητά κλήσεις tools, που ο κώδικας τρέχει και του επιστρέφει, είτε γράφει απάντηση, που ελέγχεται. Πέντε γύροι το πολύ, μία επισκευή το πολύ. Εδώ η ακολουθία εξαρτάται από την ερώτηση, και το μοντέλο είναι το μόνο που μπορεί να την αποφασίσει αφού τη διαβάσει.

Οι ερωτήσεις: βρόχος δύο κόμβων, τα tools τα διαλέγει το μοντέλο



Πέντε γύροι το πολύ: ένα μοντέλο που δεν σταματά να ζητά tools είναι 502 και δεν αφήνει τίποτα πίσω του. Η άρνηση είναι απάντηση: γράφεται στη συνομιλία.

Tool calling. Ποια πλευρά κρατά τα tools το αποφασίζει το τι διακυβεύεται. Στην προσφορά τον κατάλογο και το ευρετήριο τα καλεί ο γράφος, σε σταθερά σημεία, με ορίσματα που υπολόγισε το προηγούμενο βήμα. Στον assistant τα tools τα καλεί το μοντέλο, μέσω του tool calling του OpenAI API, με τρία εργαλεία στο `assistant/tools.py`. Το `product_stock` επιστρέφει τη γραμμή του `/products/{sku}/stock` ή, για κωδικό που δεν υπάρχει, την πρόταση «Δεν υπάρχει προϊόν με κωδικό ... στον κατάλογο», ώστε το μοντέλο να έχει κάτι αληθινό να πει. Το `find_products` επιστρέφει έως δέκα γραμμές μιας κατηγορίας κάτω από μια τιμή. Το `search_policies` επιστρέφει έως τέσσερα αποσπάσματα, και ένα απόσπασμα `datasheet` μόνο αν ο κωδικός του είναι ανάμεσα στα θέματα της συνομιλίας, αλλιώς η εγγύηση του PSU-1018 θα απαντιόταν από το φυλλάδιο του PSU-1020. Κανένα tool δεν αλλάζει τίποτα.

Grounding ως βρόχος επισκευής, όχι ως βαθμός. Τίποτα γραμμένο από το μοντέλο δεν εμφανίζεται πριν διαβαστεί απέναντι στα δεδομένα που είχε μπροστά του. Στη σύσταση, το `recommendation.py` χτίζει το σύνολο των αριθμών που η απάντηση επιτρέπεται να περιέχει, από το αίτημα, τις προσφορές και τις σημειώσεις τους, και διαβάσει το «γιατί» και το κείμενο με

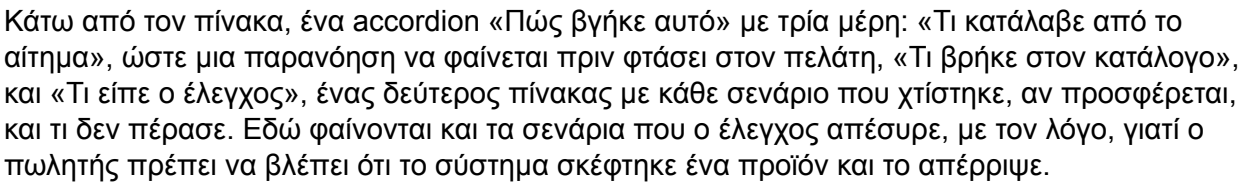
τα ποσά γραμμένα είτε όπως τα γράφει ένα JSON είτε όπως τα γράφει ένας Έλληνας, 3.656,44, ό,τι κωδικός ή αριθμός δεν βρίσκεται στο σύνολο γυρίζει στο μοντέλο με τη φράση σε εισαγωγικά, έως τρεις γύρους. Τα τμήματα πολιτικής που συνοδεύουν τις προσφορές διαλέγονται από ό,τι κουβαλούν οι προσφορές, μια έκπτωση, μια έγκριση, μια μεταφορά, όχι από τα λόγια του πελάτη. Και μια σημείωση που γράφει ο κώδικας λέει για κάθε προσφορά που είναι ακριβότερη, όχι γρηγορότερη και όχι καλύτερη από άλλη ότι δεν υπάρχει τίποτα να την προτιμήσει κανείς, ώστε τη σύγκριση να μην την κάνει το μοντέλο. Στον assistant, το assistant/grounding.py διαβάζει την απάντηση απέναντι στα αποτελέσματα των tools αυτού του γύρου με έναν διαχωρισμό: ένας κωδικός προϊόντος επιτρέπεται να έρχεται και από προηγούμενο γύρο, γιατί είναι το θέμα της κουβέντας, αλλά ένας αριθμός μόνο από την ερώτηση και από τα tools αυτού του γύρου, γιατί μια τιμή που ειπώθηκε χθες δεν είναι απόδειξη σήμερα. Μία επισκευή, και μετά άρνηση που κατονομάζει τι δεν τεκμηριώθηκε. Πουθενά δεν υπάρχει βαθμός «πόσο καλή» είναι μια απάντηση: αυτό είναι κρίση, και ένας αριθμός για αυτήν θα ήταν το μοντέλο να βαθμολογεί τη δουλειά του.

8. Το UI

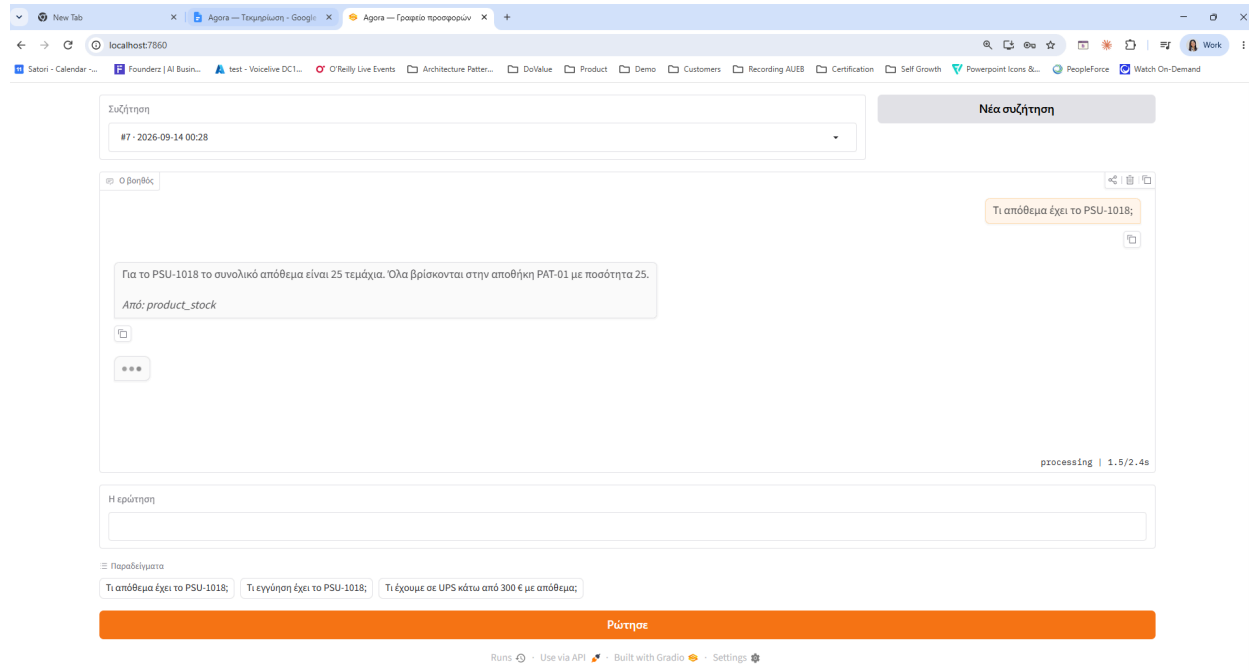
Η οθόνη, στο `ui/gradio_app.py`, ανοίγει με πύλη σύνδεσης: «Χρήστης», «Κωδικός», «Σύνδεση». Η φόρμα πηγαίνει στο `/auth/login` της υπηρεσίας προσφορών και το `token` κρατιέται στην κατάσταση του Gradio. Κάθε τι που δείχνει η οθόνη το ζήτησε μέσω HTTP.

The screenshot shows a web browser window with the URL `localhost:7860`. The page title is "Agora". Below the title, it says "Συνδέσου για να συνεχίσεις." (Log in to continue). There are two input fields: "Χρήστης" (Username) and "Κωδικός" (Password). Below these fields is an orange button labeled "Σύνδεση" (Login). At the bottom of the page, there is a footer with the text "Use via API" followed by a small icon, "Built with Gradio" followed by a small icon, and "Settings" followed by a gear icon.

Η καρτέλα «Προσφορές» είναι το γραφείο προσφορών. Πάνω, το πεδίο του αιτήματος, ένα dropdown με τα επτά καταστήματα, Αθήνα, Πειραιάς, Θεσσαλονίκη, Πάτρα, Τρίπολη, Λάρισα και Ηράκλειο, το checkbox «Παραγγελία πριν τις 13:00» και έξι έτοιμα παραδείγματα. Κάτω από το κουμπί «Ετοιμασία προσφοράς» εμφανίζεται η σύσταση: επικεφαλίδα με κωδικό, περιγραφή και σύνολο, το «Γιατί» προς τον πωλητή, το «Πρόσεξε» με τις προϋποθέσεις, και το κείμενο «Προς τον πελάτη» σε παράθεση. Από κάτω, ένας πίνακας με όλες τις προσφορές που πέρασαν τον έλεγχο, με τις στρατηγικές μεταφρασμένες σε λέξεις πωλητή, «φθηνότερο», «πιο κοντινό», «αξία χρήματος», «γρηγορότερο», «ανώτερο», και τα ποσά γραμμένα ως 1.602,63 €.



στηρίζεται να φαίνεται χωρίς να ανοίξει κανείς τίποτα. Μια άρνηση εμφανίζεται στην ίδια θέση με μια απάντηση.



9. Εγκατάσταση, εκτέλεση και παραδείγματα

Το Agora θέλει Python 3.14 και Poetry. Από τη ρίζα του αποθετηρίου, το poetry install φτιάχνει το εικονικό περιβάλλον και εγκαθιστά ό,τι κλειδώνει το poetry.lock.

```
git clone <repository-url>
```

```
cd agora
```

```
poetry install
```

Μετά αντιγράφεται το πρότυπο των μεταβλητών περιβάλλοντος σε ένα αρχείο .env στη ρίζα. Δύο τιμές χρειάζονται οπωσδήποτε: το OPENAI_API_KEY και το JWT_SECRET_KEY, για το οποίο αρκεί μια μακριά τυχαία συμβολοσειρά. Το πρότυπο γράφει LLM_MODEL=gpt-5.1, οι μετρήσεις του κεφαλαίου 10 έγιναν με το gpt-4o-mini, που είναι και η προεπιλογή του κώδικα όταν η γραμμή λείπει, οπότε όποιος θέλει να τις αναπαραγάγει βάζει εκεί gpt-4o-mini.

```
copy .env.example .env
```

```
poetry run python -c "import secrets;
```

```
print(secrets.token_urlsafe(48))"
```

Η βάση είναι committed και γεμάτη, οπότε δεν εισάγεται τίποτα. Οι δύο υπηρεσίες ξεκινούν από τη ρίζα, η καθεμία στο δικό της τερματικό. Το ευρετήριο χτίζεται μία φορά, με την υπηρεσία καταλόγου να τρέχει, και επειδή τα διανύσματα είναι committed, σε φρέσκο clone το χτίσιμο δεν κοστίζει τίποτα.

```
poetry run uvicorn services.data_service.main:app --reload --port 8001
```

```
poetry run uvicorn services.offer_service.main:app --reload --port 8000
```

```
poetry run python -m services.offer_service.rag.indexer
```

Η οθόνη σερβίρεται στο http://localhost:7860 και ζητά όνομα και κωδικό, ο κατάλογος γράφει

δύο δοκιμαστικούς λογαριασμούς την πρώτη φορά που ξεκινά με άδριο πίνακα χρηστών, και η βάση έρχεται με τους δύο λογαριασμούς ήδη μέσα, rmoschos με κωδικό ai-for-devs-moschos και amerlou με κωδικό ai-for-devs-merlou. Η ίδια ροή τρέχει από τη γραμμή εντολών, και τα τεστ και ο linter τρέχουν χωρίς κλειδί.

```
poetry run python ui/gradio_app.py
```

```
poetry run python -m tools.run_offer "Ten 80+ Gold ATX power supplies,  
at least 750W."
```

```
poetry run pytest
```

```
poetry run ruff check .
```

Παράδειγμα 1, ένα αίτημα που γίνεται προσφορές. Το αίτημα είναι «Θέλω 10 τροφοδοτικά 80+ Gold σε ATX, τουλάχιστον 750W.», με κατάσταση τη Θεσσαλονίκη. Η εξαγωγή το διαβάζει σε κατηγορία PSU, ποσότητα 10, efficiency ίσο με 80+ Gold, form_factor ίσο με ATX και watt τουλάχιστον 750, σε έναν γύρο. Με πέντε υποψήφιους ο builder χτίζει τρία σενάρια, γιατί δύο ζεύγη στρατηγικών πέφτουν στο ίδιο προϊόν.

Προϊόν	Πώς επιλέχθηκε	Απόθεμα	Παράδοση	Καθαρή αξία	Έκπτωση	Σύνολο
PSU-1018, Τροφοδοτικό 1000W 80+ Gold ATX	φθηνότερο, γρηγορότερο	10 από PAT-01, μεταφορά	4 εργάσιμες	565,70 €	0%	565,70 €
PSU-1002, Τροφοδοτικό 750W 80+ Gold ATX	πιο κοντινό, αξία χρήματος	10 κατόπιν παραγγελίας	16 εργάσιμες	702,20 €	0%	702,20 €
PSU-1006	ανώτερο	4 από ATH-02, 6 κατόπιν παραγγελίας	7 εργάσιμες	2.311,00 €	3%, 69,33 €	2.241,67 €

Τα μεταφορικά είναι μηδέν και στα τρία, γιατί και τα τρία ξεπερνούν τα 300 € καθαρής αξίας. Η παγίδα είναι ότι το φθηνότερο δεν είναι το πιο κοντινό: το PSU-1018 είναι 1000W εκεί που ζητήθηκαν 750W, οπότε παίρνει fit 0,6667 και το PSU-1002 παίρνει 1,0. Η σύσταση διάλεξε το PSU-1018, και το κείμενο προς τον πελάτη, όπως το έγραψε το μοντέλο στην τελευταία μέτρηση, είναι: «Σας προτείνουμε 10 τεμάχια από το τροφοδοτικό «Τροφοδοτικό 1000W 80+ Gold ATX» (κωδ. PSU-1018), που καλύπτει τις απαιτήσεις σας για 80+ Gold ATX και ισχύ τουλάχιστον 750W. Η συνολική αξία της προσφοράς είναι 565.7. Ο εκτιμώμενος χρόνος παράδοσης είναι 4 εργάσιμες ημέρες. Οι 4 εργάσιμες ημέρες περιλαμβάνουν ήδη τον χρόνο εσωτερικής μεταφοράς μεταξύ αποθηκών.» Το «γιατί» προς τον πωλητή συγκρίνει τα τρία σύνολα, 565.7 έναντι 702.2 και 2241.67, και κλείνει με την εναλλακτική για όποιον θέλει ακριβώς 750W. Κάθε αριθμός σε αυτές τις προτάσεις υπάρχει στον πίνακα από πάνω, αν δεν υπήρχε, το κείμενο δεν θα είχε φτάσει εδώ.

Παράδειγμα 2, μια ερώτηση που απαντιέται από tools. Η ερώτηση «Τι απόθεμα έχει το PSU-1018;» οδηγεί το μοντέλο να καλέσει το product_stock με κωδικό PSU-1018. Το tool επιστρέφει τη γραμμή του καταλόγου, 25 τεμάχια στην PAT-01 και τίποτα αλλού, η απάντηση

λέει αυτό με την αποθήκη ονομαστικά, και από κάτω γράφεται «Από: product_stock». Η ερώτηση «Τι εγγύηση έχει το PSU-1018;» οδηγεί στο search_policies, και το tool επιστρέφει το απόσπασμα «Εγγύηση και Επιστροφές, Εγγύηση ανά κατηγορία», όπου τα τροφοδοτικά έχουν τρία χρόνια. Το φυλλάδιο του PSU-1020 έχει κι αυτό ενότητα εγγύησης, αλλά δεν επιστρέφεται, γιατί το PSU-1020 δεν είναι θέμα της συνομιλίας. Στις μετρήσεις και οι δύο ερωτήσεις απαντήθηκαν με αυτά τα γεγονότα, και στις τρεις εκτελέσεις, όπως και μια συνομιλία δύο γύρων για το UPS-1021, όπου το δεύτερο ερώτημα, «και τι εγγύηση έχει;», κληρονομεί τον κωδικό από το πρώτο και απαντιέται με τα δύο χρόνια των UPS.

Παράδειγμα 3, μια ερώτηση που απορρίπτεται. Ένας κωδικός που δεν υπάρχει, «Τι απόθεμα έχει το PSU-9999;», δεν είναι άρνηση: το product_stock επιστρέφει «Δεν υπάρχει προϊόν με κωδικό PSU-9999 στον κατάλογο» και αυτή είναι η απάντηση. Άρνηση είναι ό,τι το μοντέλο έγραψε και κανένα tool δεν στηρίζει: αν σε επόμενο γύρο ξαναγράψει μια τιμή από τη μνήμη του χωρίς να ξανακαλέσει tool, ο έλεγχος τη στέλνει πίσω μία φορά, και αν η δεύτερη απάντηση έχει το ίδιο πρόβλημα, η οθόνη δείχνει τη σταθερή διατύπωση του κώδικα: «Δεν μπορώ να τεκμηριώσω αυτή την απάντηση από τον κατάλογο και τα έγγραφα: «...» δεν προέκυψε από κανένα εργαλείο. Ρώτησε πιο συγκεκριμένα, με τον κωδικό του προϊόντος αν τον ξέρεις.», με τη φράση που δεν τεκμηριώθηκε στη θέση των αποσιωπητικών.

[SCREENSHOT 5 — Η καρτέλα «Ερωτήσεις» με μια άρνηση στη θέση της απάντησης. Λεζάντα: Μια απάντηση που δεν τεκμηριώθηκε δεν φτάνει στον πωλητή, φτάνει η άρνηση.]

10. Αποτελέσματα, περιορισμοί και επόμενα βήματα

Κάθε κομμάτι που αγγίζει το μοντέλο μετρήθηκε πάνω σε σύνολο περιπτώσεων που έγραψα με το χέρι πριν τρέξω τον κώδικα, με τις αναμενόμενες απαντήσεις υπολογισμένες σε SQL ή διαβασμένες από τα έγγραφα, και κάθε μέτρηση με μοντέλο επαναλήφθηκε τρεις φορές. Τα σύνολα είναι στο tests/data και τα εργαλεία στο tools.

Μέτρηση	Σύνολο	Αποτέλεσμα
Ανάκτηση προϊόντων από την πρόταση του πωλητή, recall στα 5 / στα 10	12 αιτήματα	92% / 99%
Κατάταξη μόνη της, χωρίς το φίλτρο: λέξεις, νόημα, συγχώνευση	12 αιτήματα	48 / 66, 44 / 52, 49 / 74
Εξαγωγή απαιτήσεων ίδια με τη χειρόγραφη	12 αιτήματα, 3 εκτελέσεις	10 από 12 ακριβώς, 12 από 12 ίδιες και στις τρεις
Ανάκτηση εγγράφων, recall στα 5 / στα 10	11 ερωτήσεις	95% / 100% (λέξεις μόνο 68 / 77)
Μητρώα παράδοσης και εκπώσεων απέναντι στα έγγραφα	43 αριθμοί, 32 ημερομηνίες	43 / 43, 32 / 32
Σενάρια πεδίο προς πεδίο απέναντι στα χειρόγραφα	17 σενάρια σε 6 αιτήματα	17 / 17
Ο έλεγχος πάνω σε ό,τι	340 έλεγχοι	σιωπηλός, εκτός από τις 2

Μέτρηση	Σύνολο	Αποτέλεσμα
χτίστηκε		προβλεπόμενες εγκρίσεις
Σύσταση μέσα στις προσφορές, χωρίς επισκευή	6 αιτήματα, 3 εκτελέσεις	6 / 6 με την πρώτη, 6 / 6 ίδια επιλογή
Assistant: σωστό tool, σωστά γεγονότα, καμία απαγορευμένη αναφορά	7 περιπτώσεις, 8 γύροι, 3 εκτελέσεις	7 / 7 και στις τρεις
Κείμενα e-shop αναγνώσιμα πίσω από τους parsers	226 προϊόντα	226 / 226, μέση επικάλυψη 3% σε 4.497 ζεύγη
Τεστ	202	πράσινα, χωρίς δίκτυο

Οι αριθμοί λένε ότι το σύστημα κάνει αυτό που λέει ότι κάνει, πάνω στις περιπτώσεις που σκέφτηκα. Δεν λένε ότι είναι έτοιμο για πελάτες, και οι αδυναμίες που ξέρω είναι συγκεκριμένες. Η πρώτη είναι η μορφή των ποσών: το κείμενο του παραδείγματος 1 γράφει «565.7», γιατί το μοντέλο βλέπει τα ποσά όπως τα γράφει το JSON και η οθόνη μορφοποιεί μόνο τον πίνακα, ο πωλητής πρέπει να διορθώσει το ποσό πριν το στείλει. Η δεύτερη είναι ότι οι σημειώσεις των σεναρίων είναι στα αγγλικά και το μοντέλο τις παραθέτει αυτούσιες μέσα σε ελληνική πρόταση, όπως το «nothing to prefer this over PSU-1018: it is cheaper, at least as fast, and meets no less of the request» στο «γιατί» του ίδιου παραδείγματος. Η τρίτη είναι απόδοση: η σύσταση ξαναδιαβάζει τα δέκα PDF σε κάθε αίτημα για να διαλέξει τα τμήματα πολιτικής, αντί να κρατά τα 66 αποσπάσματα στη μνήμη. Η τέταρτη είναι ένα τεστ που αποτυγχάνει περίπου μία φορά στις δώδεκα, στην αναζήτηση εγγράφων, γιατί ο ψεύτικος embedder των τεστ δίνει ισοβαθμίες. Η πέμπτη είναι η σοβαρότερη: ο έλεγχος τεκμηρίωσης της σύστασης πιάνει κάθε αριθμό και κάθε κωδικό που δεν υπάρχει στις προσφορές, αλλά όχι μια ψευδή σύγκριση φτιαγμένη από αληθινούς αριθμούς. Το είδα να συμβαίνει σε προηγούμενη έκδοση, όταν μια προσφορά ονομάστηκε «φθηνότερη» ενώ ήταν ακριβότερη, το διόρθωσα για εκείνη την περίπτωση με μια σημείωση που γράφει ο κώδικας, αλλά γενικό επαληθευτή για τους ισχυρισμούς του «γιατί» δεν έχω γράψει. Μικρότερα: το μονολεκτικό «το φθηνότερο» αποτυγχάνει στην εξαγωγή ενώ το «το πιο φθινό» δουλεύει, και το ruff format θα ξανάγραφε 19 αρχεία. Τα επόμενα βήματα βγαίνουν από τα παραπάνω. Πρώτα ο επαληθευτής των ισχυρισμών: κάθε σύγκριση που γράφει το «γιατί» να ξαναυπολογίζεται από τις προσφορές, όπως ξαναυπολογίζονται σήμερα οι αριθμοί. Μετά η μορφοποίηση των ποσών πριν φτάσουν στο μοντέλο, ώστε να γράφει «565,70 €» επειδή αυτό διάβασε, και οι σημειώσεις στα ελληνικά. Μετά τα 66 αποσπάσματα στη μνήμη. Πιο μακριά, μια στήλη κόστους για να ελέγχεται το ελάχιστο περιθώριο 12% της πολιτικής, το ιστορικό πελάτη για την έκπτωση 2% των συμβάσεων, και το ίδιο σχήμα πάνω σε πραγματικό κατάλογο αντί για έναν που γεννήθηκε από σπόρο.

Παράρτημα Α: πώς χτίστηκε

Α.1 Η μέθοδος

Το Agora χτίστηκε ανάμεσα στις 5 και τις 13 Σεπτεμβρίου 2026, σε έξι ημέρες δουλειάς, με μία μέθοδο που έμεινε ίδια από την πρώτη μέρα και που αξίζει να γραφτεί πριν από την πορεία, γιατί εξηγεί την πορεία. Η μέθοδος έχει πέντε κανόνες. Πρώτον, τα σύνολα αξιολόγησης γράφτηκαν με το χέρι πριν από τον κώδικα που θα μετρούσαν.

Για την ανάκτηση προϊόντων έγραψα δώδεκα αιτήματα με τις απαιτήσεις τους, το καθένα με μια παγίδα, έναν κωδικό κολλημένο αυτούσιο, ένα αίτημα χωρίς κανέναν αριθμό, ένα στα αγγλικά, ένα με υπερθετικό, και υπολόγισα τις σωστές απαντήσεις με SQL πάνω στη βάση. Για τα έγγραφα, έντεκα ερωτήσεις με τα αποσπάσματα που τις απαντούν, διαβασμένα από εμένα. Για τα σενάρια, έξι αιτήματα και δεκαεπτά σενάρια με κάθε πεδίο τους, υπολογισμένα με το χέρι από τη βάση και από τους πίνακες των εγγράφων, ποτέ τρέχοντας τον builder. Για τον assistant, επτά περιπτώσεις σε οκτώ γύρους με το tool που πρέπει να κληθεί, τα γεγονότα που πρέπει να ειπωθούν και τους κωδικούς που δεν επιτρέπεται να αναφερθούν. Ένα σύνολο που γράφεται μετά τον κώδικα μετρά τι κάνει ο κώδικας, ένα που γράφεται πριν μετρά τι έπρεπε να κάνει. Δεύτερον, κάθε κανόνας κάθε prompt κρατήθηκε ή πετάχτηκε πάνω σε τρεις εκτελέσεις σε temperature 0. Το έμαθα με κόστος: οι πρώτοι γύροι δουλειάς στο prompt της εξαγωγής έγιναν στην προεπιλεγμένη θερμοκρασία και τρεις διαδοχικές μετρήσεις έδωσαν 8, 9 και 9 σωστά αιτήματα, μετά 9, 10 και 8, μετά 9, 9 και 8. Η διασπορά ανάμεσα στα σύνολα ήταν μικρότερη από τη διασπορά μέσα σε ένα σύνολο, οπότε δύο γύροι γραψίματος κανόνων κυνηγούσαν θόρυβο δειγματοληψίας και τον κατέγραφαν ως πρόοδο. Το να διαβάζεις μια πρόταση δεν είναι δημιουργική δουλειά και δεν υπήρξε ποτέ λόγος να δειγματοληπτείται. Σε temperature 0, δύο από τις τρεις εκτελέσεις γυρίζουν byte προς byte ίδιες και η τρίτη διαφέρει σε ένα αίτημα, και αυτό κάνει μια διαφορά που απομένει εύρημα αντί για ισοπαλία. Με έντεκα περιπτώσεις τότε, ένα αίτημα που αλλάζει μετακινεί τον μέσο όρο εννέα μονάδες, οπότε ό,τι αξίζει λιγότερο από αυτό δεν φαίνεται από εδώ, και το ξέρω.

Τρίτον, ένα όργανο μέτρησης δεν το πίστεψα πριν το δοκιμάσω με μεταλλάξεις. Ο έλεγχος των σεναρίων αποδείχθηκε ότι πιάνει κάτι μόνο όταν χάλασα επίτηδες ένα καθαρό σενάριο με πέντε διαφορετικούς τρόπους και είδα κάθε φθορά να πιάνεται από τον έλεγχο που της ανήκει. Το ίδιο έκανα στον έλεγχο τεκμηρίωσης της σύστασης, με έναν αριθμό που δεν υπάρχει πουθενά αλλά είναι σχεδόν σωστός, και στον έλεγχο του assistant, με μια απάντηση που δανείζεται από προηγούμενο γύρο έναν αριθμό αντί για έναν κωδικό. Και το αντίστροφο: ο έλεγχος τεκμηρίωσης διαβάστηκε και για τους ψευδείς συναγερμούς του, γιατί ένας έλεγχος που απορρίπτει σωστές απαντήσεις κοστίζει στον πωλητή μια απάντηση όπως ένας που αφήνει λάθος κοστίζει στον πελάτη έναν ισχυρισμό.

Τέταρτον, μια πράσινη σουίτα δεν αποδεικνύει τίποτα για μια αλλαγή που δεν ασκεί. Τα 202 τεστ τρέχουν χωρίς μοντέλο και χωρίς δίκτυο, και αυτό είναι η δύναμή τους και το όριό τους: δεν μπορούν να δουν αν ένα prompt έγινε χειρότερο, αν μια κατάταξη έθαψε ένα αίτημα ή αν το μοντέλο άρχισε να απευθύνεται στον πελάτη αντί στον πωλητή. Για αυτά υπάρχουν τα εργαλεία μέτρησης, χωριστά, και κάθε αλλαγή που άγγιζε το μοντέλο ή την ανάκτηση ξανάτρεχε το εργαλείο της, όχι τη σουίτα. Το πιο ακριβό εύρημα του έργου, ότι σε οκτώ εκτελέσεις της σύστασης δεν είχα διαβάσει ούτε μία πρόταση που έγραψε το μοντέλο, προέκυψε ακριβώς επειδή όλα ήταν πράσινα και όλοι οι αριθμοί επιτρεπτοί.

Πέμπτον, ένας ισχυρισμός που δεν μετρήθηκε είναι ισχυρισμός. Το έγγραφο αυτό δεν λέει «η υβριδική αναζήτηση βελτιώνει την ανάκτηση» αλλά «92% στα πέντε πάνω σε δώδεκα αιτήματα», και όταν ένας αριθμός δεν υπάρχει, όπως για το πόσο καλή είναι μια απάντηση του assistant, το λέει και αυτό. Ο ίδιος κανόνας ίσχυσε προς τα μέσα: κάθε τεχνική που μπήκε στο σύστημα με ένα επιχείρημα έπρεπε να μείνει με έναν αριθμό, και τρεις από αυτές δεν έμειναν.

A.2 Η πορεία, φάση προς φάση

Ο σκελετός και τα δεδομένα. Την πρώτη μέρα αποφάσισα τις δύο υπηρεσίες και το όριο ανάμεσά τους, έστησα τα δύο main.py με ένα /health το καθένα, και έγραψα τον γεννήτορα του καταλόγου: 226 κωδικοί από σπόρο, με τέσσερις εξαγωγές που διαφωνούσαν επίτηδες μεταξύ

τους στη γραφή του κωδικού, της τιμής και της ημερομηνίας. Μετά το μητρώο κατηγοριών, οι parsers, ο loader, ο normalizer και το pipeline, και πάνω τους το repository και τα routes των προϊόντων. Η πρώτη μέτρηση του έργου δεν αφορούσε μοντέλο: μέτρησα πόσα προϊόντα έχουν δίδυμο με ολόιδιες προδιαγραφές και βρήκα 102 από τα 226, σε 43 ομάδες, με τη μεγαλύτερη πέντε UPS. Αυτό άλλαξε το σχέδιο του ευρετηρίου πριν γραφτεί: αν οι προδιαγραφές δεν ξεχωρίζουν το 45% του καταλόγου, η κάρτα χρειάζεται ένα κείμενο που το ξεχωρίζει, και έτσι γεννήθηκε το κείμενο του e-shop ως στήλη, με το μοντέλο να το γράφει και τους parsers να το κρίνουν. Το ημερολόγιο των κλήσεων προς μοντέλο μπήκε την ίδια μέρα, πριν από την πρώτη κλήση με κόστος, και μέχρι το βράδυ και τα 226 κείμενα ήταν γραμμένα και επαληθευμένα. Την επόμενη μέρα ένα κλειδί από άλλο έργο, ξεχασμένο σε μεταβλητή περιβάλλοντος, σκίασε το κλειδί του .env, από τότε ο client καταγράφει ποιο κλειδί χρησιμοποιεί και από πού το πήρε. Η κονσόλα SQL ξεκίνησε με μαύρη λίστα απαγορευμένων λέξεων και την αντικατέστησα με τον authorizer του SQLite, γιατί μια λίστα λέξεων απαγορεύει ό,τι σκέφτηκα και ο authorizer ό,τι δεν είναι ανάγνωση.

Τα έγγραφα και το ευρετήριο. Έγραψα τα δέκα έγγραφα και τα απέδωσα σε PDF, και διαβάζοντάς τα ξανά βρήκα δύο αντιφάσεις ανάμεσά τους, που διόρθωσα πριν ευρετηριαστούν. Τα διανύσματα μπήκαν στο version control με digest ανά κείμενο, ώστε το ευρετήριο να ξαναχτίζεται χωρίς κόστος. Το όργανο της ανάκτησης χρειάστηκε διόρθωση πριν το πιστέψω: μετρούσε precision στα k και το ονόμαζε recall. Η πρώτη μέτρηση της ανάκτησης προϊόντων έδωσε 44% στα πέντε, στο τέλος εκείνης της ημέρας ήταν 64%, και ο κωδικός που κολλιέται αυτούσιος είχε πάει από 0% σε 100%.

Η γραμμή πλαισίου: δικαιολογημένη δύο φορές, αποσυρμένη δύο φορές. Κάθε κάρτα άνοιγε με μια γραμμή που έλεγε πού κάθεται το προϊόν μέσα στην κατηγορία του, «η μικρότερη διατομή (1.5mm) , η μεγαλύτερη ισχύς (3000W)», υπολογισμένη σε Python. 134 από τις 226 κάρτες την είχαν. Ο πρώτος λόγος που την έγραψα ήταν ότι δίνει στο embedding κάτι να ξεχωρίσει τις σχεδόν ίδιες περιγραφές. Όταν τον μέτρησα, πέθανε αριθμητικά: μόνο 9 από τις 134 γραμμές ήταν μοναδικές μέσα στην κατηγορία τους, και οι 43 ομάδες με ίδιες προδιαγραφές πήραν και οι 43 την ίδια γραμμή, αναγκαστικά, γιατί η γραμμή είναι συνάρτηση ακριβώς των δεδομένων που υποτίθεται ότι θα ξεπερνούσε. Αντί να τη σβήσω, ξανάγραψα τον λόγο: βάζει τα άκρα μιας κλίμακας σε λέξεις, που είναι αυτό στο οποίο πρέπει να ταιριάζει ένα αίτημα χωρίς κατώφλι, «το πιο μακρύ καλώδιο που έχετε». Πρόσθεσα μια περίπτωση για αυτό, με τα επτά καλώδια των 50 μέτρων ως απάντηση από SQL, και ο δεύτερος λόγος πέθανε κι αυτός: τα επτά σωστά βρέθηκαν στις θέσεις 4, 5, 8, 11, 18, 27 και 42, πίσω από τρία καλώδια που ταίριαζαν μόνο στο «καλώδιο» και στο «ρεύματος». Η ερώτηση έλεγε «πιο μακρύ» και η κάρτα «μήκος». Δοκίμασα να αφαιρέσω τις ελληνικές stopwords από το ερώτημα, η κατάταξη μόνη της βελτιώθηκε από 57 / 69 σε 61 / 77 και το σύστημα ολόκληρο χειρότερεψε από 88 / 95 σε 87 / 92, οπότε το απέρριψα. Αυτό που έμεινε είναι το Ordering: ένας υπερθετικός είναι ORDER BY και όχι ανάκτηση, και το αίτημα του μακρύτερου καλωδίου πήγε από 14% στα πέντε και στα δέκα σε 71% και 100%, με το 71 να είναι το ταβάνι, γιατί επτά σωστά δεν χωρούν σε πέντε θέσεις. Η αφαίρεση της γραμμής κόστισε στη λεξική κατάταξη μόνη της, από 57 / 69 σε 48 / 67, γιατί η γραμμή επαναλάμβανε λέξεις προδιαγραφών και ανέβαζε τη συχνότητά τους, μια επανάληψη ντυμένη ως ιδέα. Το σύστημα όπως τρέχει ανέβηκε από 88 / 95 σε 92 / 100 στα έντεκα αιτήματα που υπήρχαν τότε.

Η συγχώνευση, και ο κλάδος με τους δύο retrievers. Οι δύο κατατάξεις συγχωνεύονταν αρχικά με reciprocal rank fusion. Μετρημένη, η συγχώνευση δεν κέρδιζε ούτε τις λέξεις μόνες τους, 56 / 69 έναντι 57 / 69, και έθαβε το αγγλικό αίτημα για managed gigabit switch από 100% σε 0%, και στα έγγραφα την ερώτηση για την έκπτωση άμεσης εξόφλησης. Την αντικατέστησα με εναλλαγή, μία από κάθε λίστα, που εγγυάται σε ό,τι είναι πρώτο κάπου θέση όχι χειρότερη

από τη δεύτερη: τα έγγραφα ανέβηκαν από 82 / 91 σε 95 / 100 και τα προϊόντα από άκρη σε άκρη έπεσαν από 92 / 100 σε 92 / 98, δύο μονάδες που πλήρωσα εν γνώσει. Επειδή οι δύο collections αντέδρασαν διαφορετικά, έχτισα σε χωριστό κλάδο και τη μικτή λύση, fusion για τα προϊόντα και εναλλαγή για τα έγγραφα, και τη μέτρηση: δύο μονάδες πάνω στα δέκα για τα προϊόντα, τίποτα για τα έγγραφα, και το αγγλικό αίτημα θαμμένο ξανά. Την απέρριψα: δύο μονάδες σε μια στήλη δεν αγοράζουν ένα αίτημα που εξαφανίζεται.

Ο κανόνας της χαλάρωσης που απέτυχε στη μέτρησή του. Όταν το φίλτρο δεν αφήνει τίποτα, ο retriever αφαιρεί περιορισμούς. Η πρώτη εκδοχή διάλεγε την παραχώρηση που ανοίγει τα λιγότερα προϊόντα, ως την πιο συντηρητική. Πάνω σε επτά αδύνατα αιτήματα ο κανόνας πυροδοτήθηκε πέντε φορές, στις τέσσερις ακριβώς ένας περιορισμός άνοιγε κάτι, οπότε δεν διάλεγε τίποτα, και στην πέμπτη ο νικητής ήταν όποιος περιορισμός είχε τύχει να γραφτεί πρώτος από την εξαγωγή, και έτσι πέταξε το PoE από ένα switch 48 θυρών PoE managed. Ένας κανόνας που αποφασίζει με τη σειρά γραφής δεν αποφασίζει. Τον αντικατέστησα με την ένωση όλων των συνόλων που είναι μία παραχώρηση μακριά, χωρίς επιλογή.

Η εξαγωγή. Το prompt πέρασε πέντε γύρους, και ο γύρος που μέτρησε ήταν το temperature 0 της A.1. Έκλεισα το κλειδί της διάταξης σε δεκατέσσερις τιμές, αυτό κόστισε ένα αίτημα, όπου το «900» γράφτηκε δεύτερη φορά ως όριο VA, και το recall έπεσε από 92 / 98 σε 87 / 91. Το κράτησα, γιατί το όφελος ενός ανοιχτού κλειδιού δεν μπορούσε να μετρηθεί και το κόστος ενός λάθος κλειδιού ναι, και το αίτημα το ξαναβρήκα αργότερα διαχωρίζοντας την τιμή ανά τεμάχιο από τον προϋπολογισμό όλης της παραγγελίας, που έδωσε 92 / 99 και 11 από 12 ακριβώς. Μια πρόταση για το πότε το μοντέλο πρέπει να σωπαίνει τη μετέφερα από το prompt μέσα στο schema, ως περιγραφή πεδίου, και δύο αιτήματα έγιναν ασταθή σε temperature 0, γύρισε πίσω, και το τελικό είναι 12 από 12 ίδια σε τρεις εκτελέσεις και 10 από 12 ακριβώς, με τις δύο διαφορές αδρανείς.

Τα σενάρια και το ανεξάρτητο ground truth. Έγραψα το μοντέλο παράδοσης από τους Όρους Παράδοσης, τα domain models και τον builder, και μετά την αξιολόγηση με το χέρι, ποτέ τρέχοντας τον builder. Το να γράψω τη δεύτερη, ανεξάρτητη παραγωγή των ίδιων αριθμών βρήκε αμέσως δύο πράγματα που έξι unit tests και οι αναλλοίωτες δεν είχαν δει, γιατί ρωτούσαν μόνο αν ο κώδικας συμφωνεί με τον εαυτό του. Η Απτική εξυπηρετείται από δύο αποθήκες, όχι από μία, και ο κώδικας το είχε λάθος. Και η μεταφορά μεταξύ αποθηκών μπορούσε να μικρύνει την αναμονή, γιατί διάβαζα τη γραμμή του πίνακα για την αποθήκη που εξυπηρετεί τον προορισμό και όχι για αυτή που κρατά το προϊόν, και έτσι από την Πάτρα προς τη Θεσσαλονίκη έβγαιναν δύο ημέρες εκεί που η γραμμή της Πάτρας γράφει τρεις, 7 από τους 32 ελέγχους ημερομηνίας ήταν καθρέφτης του κώδικα και όχι έλεγχός του, και τους ξανάγραψα.

Ξαναδιαβάζοντας τον κώδικα βρήκα και ότι το bool('false') είναι True στην Python, που περνούσε απαρατήρητο γιατί καμία περίπτωση της αξιολόγησης δεν ήταν switch, η μόνη κατηγορία με λογικές προδιαγραφές. Το fit είχε κι αυτό ένα λάθος: μια υπέρβαση κάτω από «μεγαλύτερο από» χρεωνόταν ολόκληρη, οπότε ένα CAT7 έπαιρνε ποινή 1,00 σε αίτημα για πάνω από CAT6a. Το διόρθωσα και μέτρησα την αλλαγή πάνω σε 4.000 κατασκευασμένα αιτήματα: 212 άλλαξαν προϊόν. Η έκπτωση όγκου έλειπε εντελώς από την πρώτη εκδοχή και 9 από τα 17 σύνολα ήταν πολύ υψηλά, την πρόσθεσα μαζί με έκτη περίπτωση, γιατί καμία από τις πέντε δεν έφτανε σε δεσμευτικό ανώτατο όριο, και οι αριθμοί υπό έλεγχο έγιναν από 31 σε 43. Μια σημείωση έγραφε ότι η έκπτωση χρειάζεται έγκριση εκεί που η πολιτική λέει ότι το κλιμάκιο τη χρειάζεται, και το ξεχώρισα, με δέκα οριακές τιμές που έπιασαν τέσσερις παραναγνώσεις.

Ο έλεγχος. Η επαλήθευση ξεκίνησε με 16 ελέγχους, 12 που αποσύρουν και 4 που προειδοποιούν, και έγινε 20 όταν την ξαναδιάβασα με το ερώτημα «τι θα περνούσε από εδώ ενώ δεν θα έπρεπε». Ο διαχωρισμός σε ό,τι αποσύρει και σε ό,τι ταξιθεύει ως προειδοποίηση έγινε με ένα κριτήριο: αν ο αριθμός είναι λάθος, ο πελάτης δεν πρέπει να τον δει, αν ο αριθμός

είναι σωστός αλλά η προσφορά δεν είναι αυτό που ζητήθηκε, ο πωλητής πρέπει να το ξέρει και να αποφασίσει.

Η σύσταση, σε οκτώ γύρους των τριών. Έγραψα το prompt, τον κόμβο, τη δομημένη απάντηση και τον βρόχο τεκμηρίωσης, και το εργαλείο μέτρησης. Τα δύο πρώτα «ψευδή» ευρήματα του εργαλείου ήταν πραγματικά ελαττώματα: ο έλεγχος δεν διάβαζε το «3.656,44 €» όπως το γράφει ένας Έλληνας, και απέρριπτε την ώρα «13» και την «1» ημέρα που το μοντέλο είχε πάρει, σωστά, από το τμήμα πολιτικής. Τα γεγονότα αυτά τα έβαλα πάνω στην προσφορά, στις σημειώσεις της, ώστε να ανήκουν στο επιτρεπτό σύνολο. Μετά είδα ότι το fit, ένας αριθμός για εσωτερική χρήση, έφτανε στο μοντέλο και έμπαινε στο επιτρεπτό σύνολο ως «1» και «0» δωρεάν, και, χειρότερα, το μοντέλο τον διάβαζε: πρότεινε ένα UPS με αυτονομία 12 λεπτών στα 7.732,92 € αντί για ένα με 20 λεπτά στα 3.656,44 €, επειδή το πρώτο είχε fit 1,0. Το fit βγήκε από ό,τι βλέπει το μοντέλο. Δοκίμασα πρώτα έναν κανόνα ισοπαλίας πάνω στο fit στο prompt και τον μέτρησα: η πρόβλεψή μου για το τι θα άλλαζε αστόχησε σε δύο από τα έξι αιτήματα, γιατί ο κανόνας δεν πρόσθεσε ένα κριτήριο αλλά ξαναζύγισε όλη την ενότητα του «τι να διαλέξεις», και κόστισε στο αίτημα του UPS τη σωστή του απάντηση. Τον απέσυρα την ίδια μέρα. Το πιο ακριβό εύρημα ήρθε όταν, μετά από οκτώ εκτελέσεις με 6 από 6 επιτρεπτές απαντήσεις, κάθισα και διάβασα τις προτάσεις. Το «γιατί», που προορίζεται για τον πωλητή, απευθυνόταν στον πελάτη και στις έξι, επαναλάμβανε το κείμενο προς τον πελάτη στις πέντε, και δεν κατονόμαζε καμία εναλλακτική. Κανένας αριθμός δεν ήταν λάθος και κανένα εργαλείο δεν μπορούσε να το δει. Ξανάγραψα το prompt για δύο ακροατήρια, και μέτρησα: 6 από 6 κατονομάζουν εναλλακτική. Στην ίδια σειρά έμαθα κάτι για τη μορφή των κανόνων: η θετική εντολή «γράψε το σύνολο» τηρήθηκε 18 από 18 φορές, ενώ η αρνητική «ποτέ σε δική της πρόταση» έκανε την ταυτολογία που ήθελε να αποτρέψει χειρότερη, από 33% σε 69%, και η θετική της αντικατάσταση την έριξε στο 22%. Μετά έδεσα τον γράφο, έγραψα το εργαλείο της γραμμής εντολών και άνοιξα την οθόνη, δύο γύροι με την οθόνη ανοιχτή βρήκαν επτά πράγματα που καμία μέτρηση δεν είχε δει, και δύο περάσματα απλοποίησης αφαίρεσαν 51 και 71 γραμμές.

O assistant. Τον έγραψα τελευταίο, σε τέσσερα βήματα: τα τρία tools, ο γράφος με τους δύο κόμβους, ο έλεγχος τεκμηρίωσης με τον διαχωρισμό κωδικών και αριθμών, και τα routes με τις συνομιλίες. Η αξιολόγηση των επτά περιπτώσεων πέρασε 7 από 7 σε τρεις εκτελέσεις και κανένας απαγορευμένος κωδικός δεν αναφέρθηκε. Το ένα εύρημα ήταν ψευδής συναγερμός: μια απάντηση γραμμένη ως αριθμημένη λίστα γύρισε για επισκευή επειδή το «1.» στην αρχή της γραμμής διαβάστηκε ως αριθμός που κανένα tool δεν επέστρεψε. Η απάντηση ήταν σωστή και ο έλεγχος λάθος, και το διόρθωσα αφαιρώντας την αρίθμηση πριν διαβαστούν οι αριθμοί, με ένα τεστ που μεταλλάχθηκε για να αποδείξει ότι αποδεικνύει κάτι.

A.3 Οι αποφάσεις και τα επόμενα βήματα

Απόφαση	Γιατί
Δύο υπηρεσίες που διαβάζουν η μία την άλλη μόνο μέσω HTTP	Ο κατάλογος ελέγχεται χωρίς μοντέλο και το ευρετήριο δεν μπορεί να ξεμείνει σιωπηλά πίσω του.
SQLite committed, γεμάτη	Ένα clone τρέχει χωρίς να στήσει τίποτα και χωρίς να εισαγάγει τίποτα.
Ένα μητρώο κατηγοριών για γεννήτορα, API,	Μια νέα κατηγορία δεν μπορεί να ξεχαστεί σε

Απόφαση	Γιατί
prompt και φίλτρο	ένα από τα τέσσερα.
Η ακαταστασία στις εξαγωγές, όχι στο αρχείο	Η κανονικοποίηση δουλεύει σε ζωντανά δεδομένα και το αρχείο μένει αναγνώσιμο.
Το κείμενο του e-shop γράφεται από το μοντέλο και κρίνεται από τους parsers	Το 45% του καταλόγου έχει δίδυμα με ίδιες προδιαγραφές και χρειάζεται κάτι που να τα ξεχωρίζει.
Κάρτες χωρίς τιμή και απόθεμα	Αλλάζουν χωρίς να ξαναχτιστεί τίποτα, ένα ευρετήριο είναι φωτογραφία.
Διανύσματα στο αποθετήριο με digest ανά κείμενο	Το ευρετήριο ξαναχτίζεται χωρίς κλειδί και χωρίς κόστος.
Φίλτρο πρώτα, μετά BM25 και διάνυσμα πάνω στο ίδιο σύνολο	Ένα προϊόν κοντά στο νόημα δεν επιβιώνει μιας απαίτησης που δεν καλύπτει.
Συγχώνευση με εναλλαγή, όχι RRF	Η RRF έθαβε ένα αίτημα από 100% σε 0%, η εναλλαγή εγγυάται θέση $\leq 2n$.
Χαλάρωση ως ένωση παραχωρήσεων	Ο κανόνας «λιγότερα προϊόντα» αποφάσιζε με τη σειρά γραφής της εξαγωγής.
Υπερθετικά ως ταξινόμηση, όχι ανάκτηση	Ένα ORDER BY απαντά ακριβώς και δωρεάν, από 14% σε 71 / 100.
Ταξινόμηση κατά τιμή μετά την ανάγνωση των τιμών	Η τιμή δεν είναι στο ευρετήριο και δεν πρέπει να είναι.
Temperature 0 σε ό,τι διαβάζει ή συστήνει	Στην προεπιλογή ο θόρυβος ήταν μεγαλύτερος από τις αλλαγές που μετρούσα.
Το μοντέλο προτείνει, οι validators κρίνουν, τρεις γύροι	Ένα schema που απορρίπτει με λόγο διορθώνει, ένα που μαντεύει κρύβει.
Τιμή ανά τεμάχιο χωριστά από προϋπολογισμό παραγγελίας	Ένα αίτημα χανόταν όταν τα δύο ήταν ένα πεδίο.
Σενάρια χωρίς μοντέλο	Κάθε αριθμός που φτάνει σε πελάτη βγαίνει από κώδικα με τεστ.
Ground truth με το χέρι, ποτέ από τον builder	Η ανεξάρτητη παραγωγή βρήκε δύο λάθη που τα τεστ αντικατόπτριζαν.
Η έγκριση διαβάζεται στο κλιμάκιο, όχι στο περικομμένο ποσοστό	Έτσι το λέει η πολιτική, δέκα οριακές τιμές το επιβεβαίωσαν.
Έλεγχοι που αποσύρουν και έλεγχοι που προειδοποιούν	Λάθος αριθμός δεν φτάνει σε πελάτη, σωστός αριθμός σε λάθος προσφορά φτάνει στον πωλητή.
Το fit δεν φτάνει στο μοντέλο	Το διάβαζε και πρότεινε 12 λεπτά στα 7.732,92 € αντί για 20 στα 3.656,44 €.

Απόφαση	Γιατί
Δύο ακροατήρια στη σύσταση	Το «γιατί» απευθυνόταν στον πελάτη 6 στις 6 πριν το διαβάσω.
Τμήματα πολιτικής από ό,τι κουβαλούν οι προσφορές	Ένα αίτημα σπάνια κατονομάζει τους όρους που η απάντησή του πρέπει να εξηγήσει.
Η σύγκριση «τίποτα να την προτιμήσει» γράφεται από κώδικα	Ο έλεγχος διαβάζει αριθμούς, όχι σχέσεις, η ψευδής «φθηνότερη» είχε μόνο αληθινούς αριθμούς.
Grounding ως επισκευή, όχι ως βαθμός	Ένας βαθμός ποιότητας θα ήταν το μοντέλο να βαθμολογεί τη δουλειά του.
Στον assistant ο κωδικός δανείζεται από προηγούμενο γύρο, ο αριθμός όχι	Ο κωδικός είναι το θέμα, μια τιμή που ειπώθηκε χθες δεν είναι απόδειξη σήμερα.
Datasheets μόνο για τα θέματα της συνομιλίας	Η εγγύηση του PSU-1018 δεν απαντιέται από το φυλλάδιο του PSU-1020.
Η αρίθμηση λίστας δεν είναι αριθμός	Ο έλεγχος απέρριπτε σωστή απάντηση, οι ψευδείς συναγερμοί κοστίζουν κι αυτοί.
Η άρνηση είναι 200 και γράφεται στη συνομιλία	Είναι απάντηση, η αποτυχία του μοντέλου είναι 502 και δεν αφήνει τίποτα.
Login και στις δύο υπηρεσίες, χρήστες μόνο στη μία	Μία προέλευση για την οθόνη, μία αλήθεια για το ποιος είναι ποιος.

Ό,τι μένει ανοιχτό το θεωρώ δουλειά που σκοπεύω να κάνω, με αυτή τη σειρά. Ο επαληθευτής των ισχυρισμών του «γιατί», που θα ξαναυπολογίζει από τις προσφορές κάθε «φθηνότερο», «γρηγορότερο» και «καλύπτει περισσότερα», όπως ξαναυπολογίζονται σήμερα οι αριθμοί, είναι το κενό που ξέρω και δεν έχω κλείσει. Η μορφοποίηση των ποσών πριν το μοντέλο και οι σημειώσεις στα ελληνικά, ώστε το κείμενο προς τον πελάτη να φεύγει όπως είναι. Τα 66 αποσπάσματα στη μνήμη αντί για ανάγνωση των PDF σε κάθε σύσταση, και μία ανάγνωση καταλόγου λιγότερη στον γράφο. Το τεστ που αποτυγχάνει μία στις δώδεκα, που θέλει έναν ψεύτικο embedder χωρίς ισοβαθμίες. Το μονολεκτικό «το φθηνότερο» στην εξαγωγή. Το ruff format στα 20 αρχεία. Και δύο πράγματα που θέλουν δεδομένα που δεν έχω: μια στήλη κόστους, για να ελέγχεται το ελάχιστο περιθώριο 12%, και ιστορικό πελάτη, για την έκπτωση 2% των συμβάσεων. Μικρότερα, στον κώδικα: η στρατηγική του «ανώτερου» διαλέγει με το σύνολο και όχι με το προϊόν, ο πίνακας σύγκρισης δεν δείχνει το κόστος μεταφοράς, και το validation.py έχει μεγαλώσει αρκετά ώστε να αξίζει να σπάσει στα τέσσερα μέρη του.